

Food Insecurity Forecast Tool

— Setup & Usage Guide

How to install and run the scoring tool that forecasts ZIP-level food insecurity using the persisted 3-year lag ridge model. No retraining involved — this applies the already-fitted weights to new data.

Want to refresh the model's weights instead of just scoring with them? See the companion [Retrain & Refresh Guide](#).

Is this “plug and play”?

For repeated use, yes — once set up, scoring a new year is one command. But getting there the *first* time requires installing the repo and Python, so budget 15–20 minutes for one-time setup below.

QUICK REFERENCE — FOR ANYONE WHO'S ALREADY SET UP

```
python scripts/score_new_year.py \  
  --mmg "path\to\new_MMG_file.xlsx" \  
  --alice "path\to\alice_county.csv"
```

Output lands in `forecasts\fl_food_insecurity_forecast_<year>.csv`. Both `--mmg` and `--alice` are required — full setup and options below.

1 One-time setup

Get the code

Either clone the repository with git, or — if you're not comfortable with git — download it as a ZIP from GitHub (green “Code” button → “Download ZIP”) and unzip it anywhere on your machine.

```
git clone https://github.com/ltk1877/ftb-pro-bono.git
```

Install Python

You need Python 3.12 or newer. Check with:

```
python --version
```

If that fails or shows an older version, install Python from python.org.

Install the required packages

From the repository folder, run:

```
pip install -r requirements.txt
```

This is a one-time step — you don't need to repeat it unless the repository's requirements change.

If you see a warning or error mentioning `openpyxl` or `numexpr` ...

...it usually means an older copy of one of those packages was already on your machine from another project — a plain `pip install -r requirements.txt` won't upgrade an already-installed package on its own. Run:

```
pip install --upgrade openpyxl numexpr
```

then try the command again. `requirements.txt` now pins minimum versions for both, so a fresh install shouldn't hit this — but if you set this up before that pin was added, this is the fix.

Get the licensed source data

This part isn't in the repository, on purpose. Map the Meal Gap and ALICE data are licensed, not open data — `data/` is deliberately excluded from version control. You'll need your own access through Feeding Tampa Bay's Feeding America membership (for MMG) and unitedforalice.org (for ALICE, Florida only). This tool can't get you access to that data; it only knows what to do once you have it. Both files are required every run — there's no built-in default for either one.

2 Running the tool

Every time you have a new year's data, run this from the repository folder:

```
python scripts/score_new_year.py \  
  --mmg "path\to\new_MMG_file.xlsx" \  
  --alice "path\to\alice_county.csv"
```

That covers the common case: Florida only. The tool will print what it's doing as it goes — read those messages, especially any warnings, before trusting the output.

Required flags

FLAG	WHAT TO PASS
<code>--mmg</code> <code>"path.xlsx"</code>	The raw Map the Meal Gap workbook for the year you want to score (must have <code>ZCTA</code> and <code>County</code> sheets).
<code>--alice</code> <code>"path.csv"</code>	The Florida ALICE county CSV from unitedforalice.org . No default — always pass the current file explicitly.

Optional flags

FLAG	WHEN YOU NEED IT
<code>--source-year 2025</code>	Your MMG file contains more than one year and the tool can't tell which to use — it'll tell you the years it found if this happens.
<code>--state ""</code>	Score every state instead of just Florida.
<code>--model "path.json"</code>	Score against a model other than the production one — e.g. a candidate model produced by the Retrain & Refresh Guide, before deciding whether to promote it. Defaults to <code>models/cc/lag3_ridge.json</code> .
<code>--output "path.csv"</code>	Choose where the result is saved. Otherwise it goes to <code>forecasts\fl_food_insecurity_forecast_<year>.csv</code> automatically.

3 Reading the output

The output is a CSV with one row per ZIP code, including:

- Identifying columns — ZIP code, county, state, food bank
- The source and forecast years (e.g. 2025 data forecasting 2028)
- Every driver value the model actually used (unemployment rate, poverty rate, cost of food, etc.) — so the forecast isn't a black box
- `predicted_food_insecurity_rate` — the forecast itself

A handful of ZIPs may come through with a blank prediction. That's not a bug — Map the Meal Gap doesn't publish a rate for very small ZIPs, and the model has nothing to build a forecast from in that case. The tool prints exactly how many rows this affected.

4 If something goes wrong

MESSAGE	WHAT IT MEANS
MMG workbook not found / ALICE CSV not found	Check the file path after <code>--mmg</code> or <code>--alice</code> is correct.
the following arguments are required: <code>--alice</code>	Both <code>--mmg</code> and <code>--alice</code> must be passed on every run — there's no saved default.
is missing sheet(s) [...]	Wrong file, or a differently-formatted export — it lists the sheets it actually found so you can compare.
is missing required column(s) [...]	The file's columns don't match what the pipeline expects — MMG's export format has changed before, so this is worth double-checking against a known-good file before assuming it's user error.
contains multiple years [...]	Add <code>--source-year</code> with the year you want, from the list it prints.
Only N ZIP rows ... have complete data	Not enough usable data at that year/state to forecast responsibly — check the file actually covers what you expect.
Warning mentioning <code>openpyxl</code> or <code>numexpr</code>	See the setup callout above — <code>pip install --upgrade openpyxl numexpr</code> .

Confirmed: this tool always scores with `models/cc/lag3_ridge.json` — whatever is currently saved there, the persisted 3-year lag, ridge model. It never retrains or modifies that file itself. To refresh those weights against a new year of data — a separate, deliberate step — see the companion Retrain & Refresh Guide.